



Relatório: TAD imageRGB

Universidade de Aveiro

Dept. de Eletrónica, Telecomunicações e Informática

Gabriel Guzmán (126180) , Henrique Tardelli (125087)

14 de novembro de 2025

1 Introdução

O desenvolvimento de **Tipos Abstratos de Dados (TADs)** é fundamental no estudo de Algoritmos e Estruturas de Dados. O projeto incide sobre o TAD **imageRGB**, cuja função principal é permitir a representação e manipulação de imagens digitais cujos pixels são codificados segundo o modelo de cor RGB, utilizando 8 bits por canal.

Cada imagem é descrita internamente através de uma matriz bidimensional de inteiros, cujos elementos fazem o index a uma **Look-Up Table (LUT)** responsável por armazenar os valores RGB correspondentes às diferentes cores presentes na imagem.

O principal objetivo deste trabalho consistiu em implementar as funções essenciais para o funcionamento do TAD, incluindo as operações de cópia, comparação e rotação, e preenchimento de imagem. Em particular, foram realizados testes computacionais e uma análise formal à função `ImageIsEqual()`. As análises foram realizadas de seguinte modo: (i) **análise experimental**, baseada na contagem efetiva das comparações entre pixels em diferentes cenários; (ii) **análise formal**, baseada na determinação matemática da sua complexidade computacional nos melhores e piores casos, assim como seu caso médio.

Adicionalmente, foram implementadas e testadas três variantes do algoritmo de *Region Growing by Flood Filling*: uma versão recursiva, duas versões iterativas, uma baseada em fila (**queue**) e outra baseada em pilha (**stack**).

Este relatório apresenta e discute os resultados obtidos, incluindo a análise detalhada do comportamento da função `ImageIsEqual`, bem como a comparação crítica entre as abordagens de *Flood Filling* utilizadas. O trabalho termina com conclusões relativas ao TAD **imageRGB** e a forma com que foi implementado, junto as considerações finais do projeto.

2 Análise da função `ImageIsEqual()`

A função `ImageIsEqual()` tem como objetivo verificar se duas imagens, representadas pelo TAD **imageRGB**, possuem exatamente o mesmo conteúdo. Para tal, a função compara primeiro as dimensões das imagens e, se estas coincidirem, compara pixel a pixel os valores RGB correspondentes, acedendo aos índices da *Look-Up Table (LUT)* de cada imagem. O processo termina assim que a comparação detecta que o valor que atribuí a cor à ambos pixels são diferentes, devolvendo 0; caso todo o percurso seja concluído sem detetar qualquer diferença, a função devolve 1.

2.1 Análise experimental

A análise experimental é um método para estudar a complexidade de um algoritmo ao utilizar instancias diferentes e analisar o número de operações efetuadas. Neste caso, só analisamos experimentalmente o pior caso, utilizamos 8 instancias e revisamos o número de comparações em cada uma. A saída no terminal encontram-se em anexos Figura 2. No gráfico Figura 1 observa-se um crescimento linear da complexidade o qual comprova-se ao calcular o resultado entre $T(2n)/T(n)$. Sendo assim, a função `IsEqual()` foi enquadrada como $O(n)$ sendo n o número de pixels da imagem.

Pixels (n)	Comparações	$T(2n)/T(n)$
200	202	1.990
400	402	1.995
800	802	1.997
1600	1602	1.998
3200	3202	1.999
6400	6402	1.999
12800	12802	1.999
25600	25602	

Tabela 1: Tabela com Pixels, Comparações, tempos e razão $t(2n)/t(n)$.

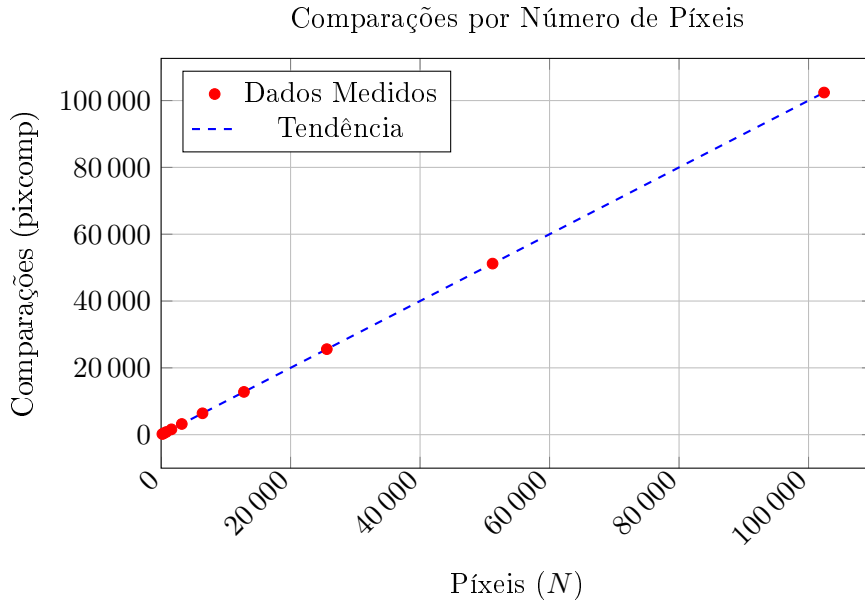


Figura 1: Gráfico com escala ajustada y menor espacio à direita.

2.2 Análise Formal

A análise formal permite determinar o número de operações realizadas por uma função independentemente de qualquer medição empírica, a operações utilizadas para medir a complexidade do algoritmo foram as comparações.

Primeiro, a função verifica se as dimensões das duas imagens são iguais. Esta operação tem custo constante, realizando um número fixo de comparações:

$$O(1).$$

Se as dimensões forem diferentes, a função termina imediatamente, sendo este o **melhor caso**, com apenas uma comparação relevante.

Quando as imagens possuem as mesmas dimensões, o algoritmo percorre toda a matriz de pixels, realizando duas operações principais por posição: leitura dos índices dos dois pixels e comparação dos valores RGB correspondentes nas LUT, e caso ambas imagens sejam iguais ele deve percorrer todos os pixels e realizar as operações anteriores, sendo este o **pior caso** (Do ponto de vista de esforço computacional).

2.2.1 Melhor caso

O melhor caso da complexidade do algoritmo consiste em quando o atributo *height* de ambas imagens é diferente ou ambos atributos *height* e *width* são diferentes, em ambos casos realizará apenas **1** operação de comparação, no caso de *height* ser igual mas *width* ser diferente em ambas imagens, este **não é considerado melhor caso** já que neste caso serão realizadas 2 comparações por ter que comparar os atributos *width* de ambas imagens condições com *OR*. Logo o **melhor caso** será dado pela seguinte fórmula:

$$B(n) = 1$$

2.2.2 Pior caso

O pior caso a ser considerado para o algoritmo é se ambas as imagens forem iguais, pois será necessário percorrer todos os píxeis e compará-los um a um. Poderíamos também considerar o caso em que apenas o último píxel é diferente, mas o número total de comparações realizadas seria exatamente o mesmo. A fórmula para o **pior caso** é dada por:

$$2 + \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} 1,$$

onde os termos significam:

- o 2 representa a verificação inicial das dimensões das imagens, que compara altura e largura;
- o somatório duplo percorre todas as posições (i, j) da imagem, e o termo 1 representa o custo constante da comparação realizada em cada iteração.

Simplificando a soma dupla obtemos

$$\sum_{i=0}^{H-1} \sum_{j=0}^{W-1} 1 = \sum_{i=0}^{H-1} W = H \times W.$$

Substituindo na expressão inicial:

$$2 + \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} 1 = 2 + H \times W.$$

Como a constante 2 é insignificante para grandes imagens, o número total de comparações no pior caso pode ser escrito como

$$W(n) = H \times W \in O(n),$$

onde $n = H \times W$ representa o número total de píxeis da imagem. Assim, o custo é linear no tamanho da entrada, já que cada píxel é visitado exatamente uma vez.

Para compreender melhor este resultado, observemos resumidamente o que ocorre dentro dos ciclos aninhados. Após verificar que as dimensões das imagens coincidem, o algoritmo percorre toda a matriz de píxeis. Em cada posição (i, j) , é feita uma única comparação entre as cores dos píxeis, contribuindo com uma operação por iteração.

No pior caso, como as imagens são iguais, nenhuma comparação termina antecipadamente e o algoritmo executa as $h \times w$ iterações completas, realizando exatamente esse número de comparações.

Assim, o comportamento do algoritmo no pior caso é linear, pois a quantidade de operações é dada por $H \times W$. Portanto, a complexidade do **pior** caso do algoritmo é:

$$O(n).$$

2.3 Comparação: análise experimental vs formal

Após recolha dos dados experimentais, esta secção comparará os valores medidos com os previstos teoricamente. Espera-se observar que:

- o número de comparações no pior caso cresce proporcionalmente ao número de pixels;
- o melhor caso ocorre quando existe uma discrepância logo no início da imagem;
- os valores experimentais aproximam-se das previsões formais, confirmando o comportamento linear.

3 Comparação das estratégias de Region Growing by Flood Filling

O preenchimento de regiões (*Region Growing by Flood Filling*) é uma função fundamental para segmentação de imagens, permitindo identificar áreas conexas que partilham a mesma cor. No âmbito deste trabalho, foram desenvolvidas e testadas três estratégias distintas para realizar este preenchimento:

- **Versão recursiva**
- **Versão iterativa com STACK**
- **Versão iterativa com QUEUE**

Embora todas produzam o mesmo resultado funcional, cada abordagem apresenta características diferentes em termos de desempenho, comportamento e forma de implementação. Segue-se uma análise sucinta dos principais aspetos observados em cada versão.

3.1 Abordagem recursiva

A abordagem recursiva possui uma implementação bastante direta, seguindo exatamente a definição clássica do algoritmo: para cada pixel visitado, a função chama-se recursivamente sobre os quatro vizinhos. Esta simplicidade torna o código claro e expressivo, mas traz uma limitação importante relacionada com a gestão da memória. Como cada chamada recursiva deve guardar o endereço de retorno no topo da *stack*, regiões grandes podem gerar um número elevado de chamadas aninhadas, eventualmente provocando um *stack overflow*. Assim, embora eficaz para imagens pequenas ou regiões limitadas, esta abordagem torna-se pouco robusta quando a área a preencher é extensa.

3.2 Abordagem iterativa com STACK

A versão iterativa baseada numa **stack** imita o comportamento da recursiva, substituindo as chamadas aninhadas por operações explícitas de *push* e *pop*. Isso elimina o risco de sobrecarregar a pilha do sistema, já que a estrutura utilizada é alocada com tamanho suficiente para comportar qualquer região. O algoritmo inicia-se lendo a cor original do pixel inicial; caso esta já corresponda ao novo *label*, o processo termina imediatamente. Caso contrário, o pixel é empurrado para a pilha, e o ciclo principal passa a desempilhar um pixel de cada vez, seguindo a ordem LIFO (*last-in, first-out*). Se o pixel é válido e apresenta a cor original, é pintado e os seus quatro vizinhos são colocados na pilha. Esta

estratégia reproduz o comportamento em profundidade da recursiva, mas com maior controle e segurança, evitando chamadas sucessivas que poderiam comprometer a estabilidade do programa.

3.3 Abordagem iterativa com QUEUE

A versão que utiliza uma **queue** implementa o preenchimento de forma nivelada, processando primeiro os vizinhos imediatos do pixel inicial e apenas depois os vizinhos desses, caracterizando uma expansão típica de *breadth-first search* (BFS). Após ler a cor original do pixel inicial, este é colocado na fila. O ciclo principal retira sempre o próximo elemento na ordem FIFO (*first-in, first-out*) e verifica se o pixel permanece dentro da imagem e se mantém a cor original. Caso se confirme, o pixel é pintado e os seus quatro vizinhos diretos são adicionados à fila. A validação ocorre apenas quando o vizinho é retirado da fila, o que torna o código simples e o processo de expansão mais uniforme. Esta estratégia tende a gerar uma propagação mais regular, adequada a regiões largas e distribuídas.

3.4 Conclusão da comparação

No contexto deste trabalho, a abordagem **recursiva** é simples de implementar, mas limitada por questões de memória e adequada apenas a regiões pequenas. A versão com **STACK** apresenta o comportamento mais equilibrado, mantendo a lógica da recursiva mas sem riscos de *stack overflow*, oferecendo maior segurança e desempenho. Já a abordagem com **QUEUE** difere principalmente na ordem de exploração, expandindo a região de forma mais uniforme e sendo particularmente eficaz em áreas extensas. Em suma, embora todas produzam o mesmo resultado, a escolha da estrutura de dados influencia diretamente a eficiência e robustez do algoritmo.

4 Conclusões

O desenvolvimento do TAD **imageRGB** permitiu consolidar conceitos fundamentais de Algoritmos e Estruturas de Dados, através da implementação de operações sobre imagens representadas com matrizes e tabelas de cores (*LUT*). A construção das funções de cópia, comparação e rotações evidenciou a importância do acesso eficiente a estruturas bidimensionais, bem como o impacto que esse acesso tem no desempenho. A análise da função `ImageIsEqual()`, em conjuntos com testes experimentais e estudo formal da complexidade, mostrou claramente como o padrão de comparação de pixels determina o número de operações e o comportamento nos diferentes casos.

Em seguida, a implementação das estratégias de *Region Growing by Flood Filling* — recursiva, com **stack** e com **queue** — permitiu compreender como a escolha da estrutura de dados altera a forma com que a memória é gerida, e quão eficiente é neste quesito. A comparação entre estas abordagens reforçou a ideia de que decisões estruturais são de extrema importância para o desempenho do programa.

No conjunto, o trabalho sobre o TAD **imageRGB** proporcionou uma aplicação prática dos princípios até então lecionados em AED, aprofundando a compreensão sobre estruturas de dados, análise de complexidade e técnicas fundamentais para o desenvolvimento de programas.

5 Anexos

```
=== ANALISIS FORMAL DE ImageIsEqual ===
```

Pixels(n)	Time	CalTime	pixmem	pixcomp
200	0.000001	0.000001	400	202
Pixels(n)	Time	CalTime	pixmem	pixcomp
400	0.000001	0.000000	800	402
Pixels(n)	Time	CalTime	pixmem	pixcomp
800	0.000001	0.000001	1600	802
Pixels(n)	Time	CalTime	pixmem	pixcomp
1600	0.000001	0.000001	3200	1602
Pixels(n)	Time	CalTime	pixmem	pixcomp
3200	0.000002	0.000002	6400	3202
Pixels(n)	Time	CalTime	pixmem	pixcomp
6400	0.000004	0.000003	12800	6402
Pixels(n)	Time	CalTime	pixmem	pixcomp
12800	0.000008	0.000005	25600	12802
Pixels(n)	Time	CalTime	pixmem	pixcomp
25600	0.000016	0.000011	51200	25602
Pixels(n)	Time	CalTime	pixmem	pixcomp
51200	0.000063	0.000042	102400	51202
Pixels(n)	Time	CalTime	pixmem	pixcomp
102400	0.000117	0.000077	204800	102402

Figura 2: Captura dos valores obtidos no terminal.